

Author: Louis Cheng

<https://www.linkedin.com/in/louis-cheng/>

<https://github.com/chaoyi-louis-cheng/>

Introduction

This document provides a brief technical walkthrough of my Anti-Money Laundering code portfolio hosted at: <https://github.com/chaoyi-louis-cheng/>

It's aimed at tech recruiters, potential colleagues (devs/consultants/architects) and hiring managers who'd like to assess my technical architecture and coding abilities.

The following is covered for each repository/program:

- Screenshots of the typical happy flow
- Code overview of specific features

Since this is demo code, certain liberties were taken to expedite development (e.g. hardcoding values). The relevant areas have been clearly marked using comments.

If you wish to run the code yourself, it's GPL licensed and freely downloadable from GitHub. Setup and execution instructions are provided in the respective repository's README.md file.

Portfolio Summary

My Anti-Money Laundering code portfolio consists of the following GitHub repositories:

- **aml001_complete_solution:** https://github.com/chaoyi-louis-cheng/aml001_complete_solution

A complete AML name screening solution with ETL loader, WC list loader and name screener. This is **the main repository you should review**.

- This adds an end-to-end batch process to the various other modules below, bringing all the functionality together as a coherent workflow.
- Alternatively, name screening can be performed via JSON & RESTful API calls.

- **aml002_etlloader:** https://github.com/chaoyi-louis-cheng/aml002_etlloader

The ETL loader module packaged separately. This implements several common features similar to commercial solutions:

- CSV file streaming to minimize memory use
- Configurable CSV to DB column mappings via an external JSON properties file
- Configurable validation for each input column via custom validator classes
- Bad records are output separately so they can be easily re-ingested

- **aml003_load_worldcheck_todb:** https://github.com/chaoyi-louis-cheng/aml003_load_worldcheck_todb

The World-Check list loader module packaged separately. This uses data streaming, lxml and pandas libraries to efficiently parse the World-Check list and save it to DB.

- **aml004_namescreening:** https://github.com/chaoyi-louis-cheng/aml004_namescreening

The name screening module packaged separately. This implements name matching using various algorithms (Levenshtein, token sort, fuzzy etc) similar to commercial solutions.

Tools Used

The following tools were used:

- Python 3.13 and PyCharm
- FastAPI/Uvicorn to run RESTful APIs
- MySQL for DB access

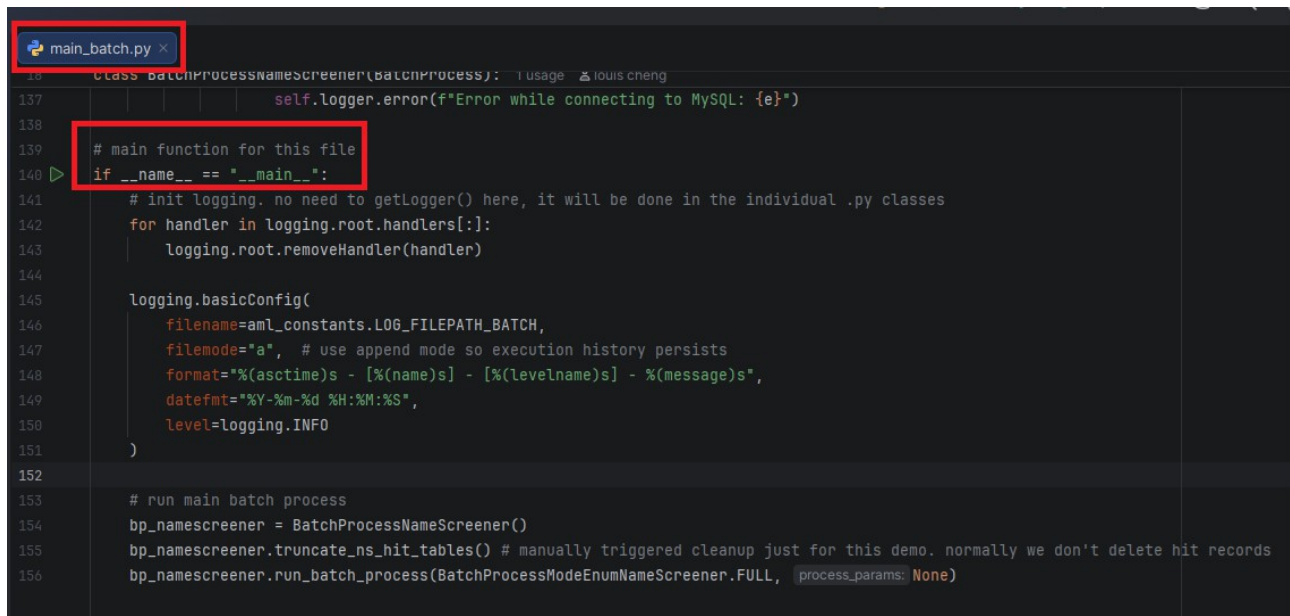
Happy Flow Overview

The following happy flow overview uses the code from the “aml001_complete_solution” repository.

We'll be running the main batch process; i.e. the “src/aml003_namescreening/main_batch.py” file.

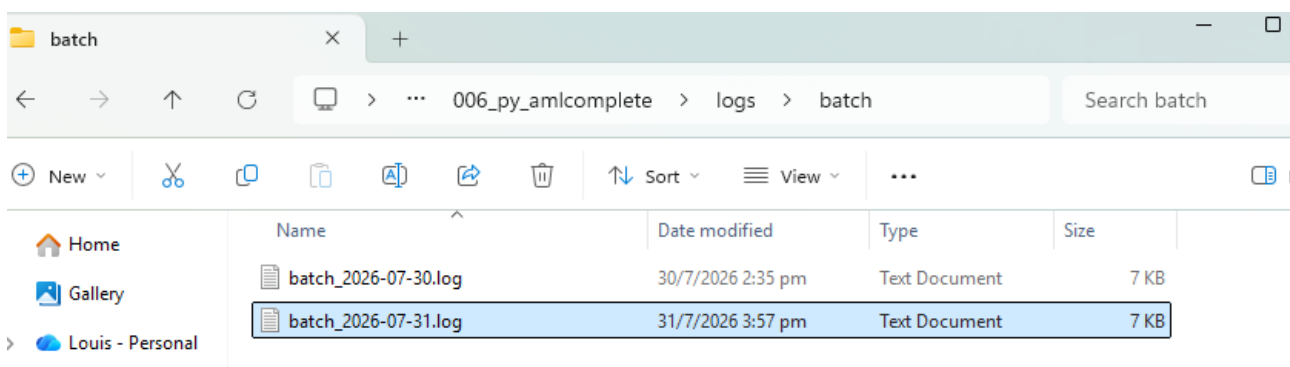
AML Complete Solution (Batch Process)

The main function for “main_batch.py” performs basic init functions like logging which must be done in the main module. It then creates a BatchProcessNameScreener and starts the main name screening batch process.



```
137 class BatchProcessNameScreener(BatchProcess):
138     self.logger.error(f"Error while connecting to MySQL: {e}")
139
140 # main function for this file
141 if __name__ == "__main__":
142     # init logging. no need to getLogger() here, it will be done in the individual .py classes
143     for handler in logging.root.handlers[:]:
144         logging.root.removeHandler(handler)
145
146     logging.basicConfig(
147         filename=aml_constants.LOG_FILEPATH_BATCH,
148         filemode="a", # use append mode so execution history persists
149         format="%(asctime)s - [(name)s] - [(levelname)s] - %(message)s",
150         datefmt="%Y-%m-%d %H:%M:%S",
151         level=logging.INFO
152     )
153
154     # run main batch process
155     bp_namescreener = BatchProcessNameScreener()
156     bp_namescreener.truncate_ns_hit_tables() # manually triggered cleanup just for this demo. normally we don't delete hit records
157     bp_namescreener.run_batch_process(BatchProcessModeEnumNameScreener.FULL, process_params=None)
```

When first triggered, the AML batch process will create a log file with today's date as the filename. All output from the process will be written to this log.

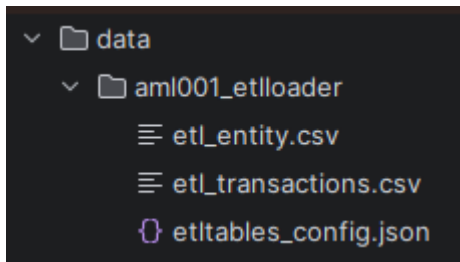


If you examine the run_batch_process function, it is comprised of:

- ETL loader step – loads the entity/transaction CSV data into the DB
- World-Check list loader step – loads the World-Check list data into the DB
- Name screener step:
 - Loads the World-Check list into memory from DB
 - Loads entities into memory from DB
 - Performs name screening on the entities

ETL Loader

Sample entity/transaction CSV is available in the “data” subfolder. A separate JSON file holds configurable column mappings for each CSV and database table. This lets the loader know which DB column a CSV column value should be written to.



The ETL loader validates the JSON file and loads the mappings into memory. From the log:

```
2026-07-31 15:57:57 - [_main_] - [INFO] - Main batch - loading entity and transaction data from CSV into database.
2026-07-31 15:57:57 - [src.aml001_etlloader.etlloader] - [INFO] - Logging engine initialized for src.aml001_etlloader.etlloader.
2026-07-31 15:57:57 - [src.aml001_etlloader.etlloader] - [INFO] - JSON table mapping configuration was successfully validated.
2026-07-31 15:57:57 - [src.aml001_etlloader.etlloader] - [INFO] - JSON table mapping configuration was successfully validated.
2026-07-31 15:57:57 - [src.aml001_etlloader.etlloader] - [INFO] - JSON configuration file at 'D:\dev\projects\001_python\006_py_amlcomplete\data\aml001_etlloader\etltables_config.json' was successfully loaded.
2026-07-31 15:57:57 - [src.aml001_etlloader.etlloader] - [INFO] - Table mapping configs: [{'table_name': 'etl_entity', 'column_mappings': [{'csv_column': 'csv_entity_id', 'db_column': 'entity_id', 'validator': 'string', 'required': True, 'primary_key': True}, {'csv_column': 'csv_entity_type', 'db_column': 'entity_type', 'validator': 'string', 'required': True}, {'csv_column': 'csv_entity_name', 'db_column': 'entity_name', 'validator': 'string', 'required': True}], ['table_name': 'etl_transactions', 'column_mappings': [{'csv_column': 'csv_tran_id', 'db_column': 'tran_id', 'validator': 'string', 'required': True, 'primary_key': True}, {'csv_column': 'csv_tran_type', 'db_column': 'tran_type', 'validator': 'string', 'required': True}, {'csv_column': 'csv_tran_amount_float', 'db_column': 'tran_amount_float', 'validator': 'float', 'required': True}, {'csv_column': 'csv_tran_currency', 'db_column': 'tran_currency', 'validator': 'string', 'required': True}, {'csv_column': 'csv_tran_datetime', 'db_column': 'tran_datetime', 'validator': 'datetime', 'required': True}, {'csv_column': 'csv_tran_ordb', 'db_column': 'tran_ordb', 'validator': 'boolean', 'required': True}], ['table_name': 'etl_transactions', 'column_mappings': [{'csv_column': 'csv_tran_id', 'db_column': 'tran_id', 'validator': 'string', 'required': True, 'primary_key': True}, {'csv_column': 'csv_tran_type', 'db_column': 'tran_type', 'validator': 'string', 'required': True}, {'csv_column': 'csv_tran_amount_float', 'db_column': 'tran_amount_float', 'validator': 'float', 'required': True}, {'csv_column': 'csv_tran_currency', 'db_column': 'tran_currency', 'validator': 'string', 'required': True}, {'csv_column': 'csv_tran_datetime', 'db_column': 'tran_datetime', 'validator': 'datetime', 'required': True}, {'csv_column': 'csv_tran_ordb', 'db_column': 'tran_ordb', 'validator': 'boolean', 'required': True}]]
```

Then, it processes each table in turn. Entities:

```
2026-07-31 15:57:57 - [src.aml001_etlloader.etlloader] - [INFO] - Initializing memory stream targeting table 'etl_entity' using source file: D:\dev\projects\001_python\006_py_amlcomplete\data\aml001_etlloader\etl_entity.csv
2026-07-31 15:57:57 - [src.aml001_etlloader.etlloader] - [INFO] -
=====
RUN SUMMARY FOR TARGET TABLE: ETL_ENTITY
=====
CSV Input File: D:\dev\projects\001_python\006_py_amlcomplete\data\aml001_etlloader\etl_entity.csv (Encoding=UTF-8)
Total Rows Processed: 5
Successful Rows: 5
Failed/Skipped Rows: 0
DB Operational Delta: 0
Error Export Log: N/A - No Errors Thrown
=====
2026-07-31 15:57:57 - [src.aml001_etlloader.etlloader] - [INFO] - No errors encountered for table 'ETL_ENTITY'. The empty error file has been cleaned up.
```

The entity data has been written to DB:

SELECT * FROM ETL_ENTITY;

	entity_id	entity_type	entity_name
▶	ENTITYID_TEST0001	CUSTOMER	TEST CUSTOMER NAME 0001
	ENTITYID_TEST0002	CUSTOMER	TEST CUSTOMER NAME 0002
	ENTITYID_TEST0003	EMPLOYEE	TEST EMPLOYEE NAME 0003
	ENTITYID_TEST0004	CUSTOMER	EVDOMI DEKATI NOEMVRI NOEMVRI DEKATI
	ENTITYID_TEST0005	EMPLOYEE	ASALA RM
✱	NULL	NULL	NULL

Transactions:

```
2026-07-31 15:57:57 - [src.aml001_etlloader.etlloader] - [INFO] - Initializing memory stream targeting table 'etl_transactions' using source file:
D:\dev\projects\001_python\006_py_amlcomplete\data\aml001_etlloader\etl_transactions.csv
2026-07-31 15:57:57 - [src.aml001_etlloader.etlloader] - [INFO] -
=====
RUN SUMMARY FOR TARGET TABLE: ETL_TRANSACTIONS
=====
CSV Input File: D:\dev\projects\001_python\006_py_amlcomplete\data\aml001_etlloader\etl_transactions.csv (Encoding=UTF-8)
Total Rows Processed: 3
Successful Rows: 3
Failed/Skipped Rows: 0
DB Operational Delta: 0
Error Export Log: N/A - No Errors Thrown
=====
2026-07-31 15:57:57 - [src.aml001_etlloader.etlloader] - [INFO] - No errors encountered for table 'ETL_TRANSACTIONS'. The empty error file has been cleaned up.
2026-07-31 15:57:57 - [_main_] - [INFO] - Main batch - loading of entity and transaction data into database is complete.
```

```
SELECT * FROM ETL_TRANSACTIONS;
```

World-Check List Loader

- data
 - aml001_etlloader
 - aml002_loadwc_todb
 - premium-world-check-small.xml
 - premium-world-check-small.xml.zip

```
2026-07-31 15:57:57 - [__main__] - [INFO] - Main batch - loading WC data from file into database.
2026-07-31 15:57:57 - [src.aml002_loadwc_todb.wcdataloader] - [INFO] - Logging engine initialized for src.aml002_loadwc_todb.wcdataloader.
2026-07-31 15:57:57 - [src.aml002_loadwc_todb.wcdataloader] - [INFO] - Truncating database tables...
2026-07-31 15:57:57 - [src.aml002_loadwc_todb.wcdataloader] - [INFO] - All WC database tables have been truncated.
2026-07-31 15:57:57 - [src.aml002_loadwc_todb.wcdataloader] - [INFO] - MySQL cursor is closed.
2026-07-31 15:57:57 - [src.aml002_loadwc_todb.wcdataloader] - [INFO] - MySQL connection is closed.
2026-07-31 15:57:57 - [src.aml002_loadwc_todb.wcdataloader] - [INFO] - Populating name type details to database...
2026-07-31 15:57:57 - [src.aml002_loadwc_todb.wcdataloader] - [INFO] - Name type details have been populated.
2026-07-31 15:57:57 - [src.aml002_loadwc_todb.wcdataloader] - [INFO] - Loading World-Check file to database...
2026-07-31 15:57:57 - [src.aml002_loadwc_todb.wcdataloader] - [INFO] - Parsing XML records from file...
2026-07-31 15:57:57 - [src.aml002_loadwc_todb.wcdataloader] - [INFO] - Records processed: 3, creating DataFrames
2026-07-31 15:57:58 - [src.aml002_loadwc_todb.wcdataloader] - [INFO] - Saving to WC_ENTITY
2026-07-31 15:57:58 - [src.aml002_loadwc_todb.wcdataloader] - [INFO] - Saving to WC_NAMES
2026-07-31 15:57:58 - [__main__] - [INFO] - Main batch - loading of WC data into database is complete.
2026-07-31 15:57:58 - [__main__] - [INFO] - Main batch - Initializing name screener and loading WC list into memory.
```

The name screener then loads all WC list entries and entities into memory for screening.

The hits and hit details are then persisted in the database. Hits:

```
SELECT * FROM NS_HITS;
```

[illegible]

Hit details:

*SELECT * FROM NS_HITDETAILS;*

	hitdetail_id	hit_id	name_id	matched_score
▶	61	24	907	85.5
	62	24	908	85.5
	63	24	920	85.5
	64	24	909	64.29
	65	24	910	64.29
	66	24	902	54
	67	24	903	54
	68	24	912	53.77
	69	24	913	53.77
	70	25	935	100
	71	25	931	90
	72	25	934	85.5
	73	25	936	85.5

If you run “main_api.py”, you'll be able to query the name screening feature via API as well.

The Postman result is shown below:

The image displays the Postman interface for an API request. The top section shows the URL `http://localhost:8000/ns/screen_ratio` and the method `POST`. The request body is `http://localhost:8000/ns/screen_ratio?input_string=DEKATI EVDOMI NOEMVRI`. The bottom section shows the JSON response, which includes a list of matched name details. A red box highlights the `matched_name_details` array, and a red note indicates that there are multiple matched names on the World-Check list.

Request:

- URL: `http://localhost:8000/ns/screen_ratio`
- Method: `POST`
- Body: `http://localhost:8000/ns/screen_ratio?input_string=DEKATI EVDOMI NOEMVRI`

Response:

```
{
  "input_string": "DEKATI EVDOMI NOEMVRI",
  "screening_type": "Match By Levenshtein Distance Ratio",
  "screening_status": "Success",
  "matched_name_details": [
    {
      "name_id": 907,
      "wc_uid": 1,
      "nametype_id": 2,
      "nametype_value": "Alias",
      "name_value": "DEKATI EVDOMI NOEMVRI",
      "matched_score": 100.0
    },
    {
      "name_id": 908,
      "wc_uid": 1,
      "nametype_id": 2,
      "nametype_value": "Alias",
      "name_value": "DEKATI EVDOMI NOEMVRI",
      "matched_score": 100.0
    },
    {
      "name_id": 909,
      "wc_uid": 1,
      "nametype_id": 2,
      "nametype_value": "Alias",
      "name_value": "DEKATI EVDOMI NOEMVRI",
      "matched_score": 85.71
    }
  ]
}
```

Notes:

- RESTful API endpoint and method
- Sends the request to the API
- A JSON result was returned
- multiple matched names on the World-Check list

This concludes the happy flow overview of a typical AML name screening process.

Code Overview

This section provides a quick peek at some of the features and their implementation details.

AML Complete Solution (Batch Process)

The main batch process is implemented as `BatchProcessNameScreener.run_batch_process()`.

The name screening functions are implemented as:

- `BatchProcessNameScreener.screen_batch()`
- `NameScreener.save_ns_hit()`

ETL Loader

CSV File Streaming

This was implemented to support large CSV files and minimize memory use. The CSV file is streamed into memory row-by-row, instead of being loaded all at once.

See `ETLLoader.load_csv_file()` for details.

```
# stream table data from CSV into memory
self.logger.info(f"Initializing memory stream targeting table '{target_table}' using source file: {csv_filepath}")

with open(csv_filepath, mode="r", encoding=csv_encoding_type) as csv_infile, \
    open(error_filepath, mode="w", encoding=self.DEFAULT_ERROR_ENCODING_TYPE, newline="") as error_outfile:

    csv_reader = csv.DictReader(csv_infile)
    error_writer = None
```

Configurable CSV To DB Column Mappings

This feature makes it very easy to customize mappings if your CSV or database columns change. It's implemented via an external JSON file – simply modify the JSON values to adjust the configuration.

See `ETLLoader.init_json_config_file` in “`etlloader.py`” for details.

The column mappings are used in `ETLLoader.persist_batch()` to construct the insertion SQL query.

```
def persist_batch(self, target_table:str, target_column_mappings:list, current_batch:list[dict]) -> int: 2+ usages ① louis cheng
    # this method returns the number of rows inserted
    # note: we re-create the DB connection for each batch
    rows_inserted = 0

    if not current_batch:
        return 0

    with get_amldb_connection() as db_connection:
        with db_connection.cursor() as db_cursor:
            try:
                # construct SQL INSERT query string
                db_cols = [mapping[self.JSON_TAG_DBCOLUMN] for mapping in target_column_mappings]
                csv_cols = [mapping[self.JSON_TAG_CSVCOLUMN] for mapping in target_column_mappings]
                primarykey_cols = []
                update_cols = []
```

Configurable Validation For Each CSV Column

This feature makes it very easy to customize validation for your data columns.

Custom validators are implemented in “etlloader.py” at the beginning of the file. Search for the “ETLBaseValidator” class. Also note the custom ETL exception class, and the validator registry.

```
VALIDATOR_REGISTRY = {
    ETLBaseValidator.validator_key_str: ETLBaseValidator(),
    ETLEmailValidator.validator_key_str: ETLEmailValidator(),
    ETLAgeRangeValidator.validator_key_str: ETLAgeRangeValidator(),
    ETLStringValidator.validator_key_str: ETLStringValidator(),
    ETLIntegerValidator.validator_key_str: ETLIntegerValidator(),
    ETLFloatValidator.validator_key_str: ETLFloatValidator(),
    ETLBooleanValidator.validator_key_str: ETLBooleanValidator(),
    ETLDateTimeValidator.validator_key_str: ETLDateTimeValidator()
}
```

If new validation logic is required, simply:

- Create a new validator subclass.
- Implement the logic within subclass.validate().
- Add the validator subclass to the VALIDATOR_REGISTRY.

Output Bad Records Separately

This feature allows bad records to be easily re-ingested.

It's implemented in ETLLoader.load_csv_file, near the end of the function.

```
if csv_row_is_valid:
    # add the valid row to the current batch
    current_batch.append(csv_row)
    success_count += 1

    # persist the current batch if it's full
    if len(current_batch) >= self.DEFAULT_PERSIST_BATCH_SIZE:
        self.logger.info(f"Current batch size of {len(current_batch)} is greater than {self.DEFAULT_PERSIST_BATCH_SIZE}, persisting batch.")
        db_persisted_total += self.persist_batch(target_table, target_column_mappings, current_batch)
        current_batch.clear()
        self.logger.info(f"Batch successfully persisted and batch size reset!")
    else:
        # handle error rows in this block
        failed_count += 1
        self.logger.warning(f"[{target_table.upper()}] Skipping row index {total_processed}, error message: {csv_row_error_msg}")

    # initialize the error writer if this is the first error
    if error_writer is None:
        error_fieldnames = list(csv_reader.fieldnames) + [self.DEFAULT_ERROR_VALIDATIONMSG_FIELDNAME] if csv_reader.fieldnames else [self.DEFAULT_ERROR_VALIDATIONMSG_FIELDNAME]
        error_writer = csv.DictWriter(error_outfile, fieldnames=error_fieldnames)
        error_writer.writeheader()

    # write the error to the output file
    csv_row[self.DEFAULT_ERROR_VALIDATIONMSG_FIELDNAME] = csv_row_error_msg
    error_writer.writerow(csv_row)
    # write the error rows to a separate file
```

World-Check List Loader

The World-Check list loader is implemented in “wcdataloader.py”.

WCDataLoader.load_wcfile_to_db_internal contains most of the parsing/extraction and persistence logic. It's a good example of XML file parsing from a stream to retrieve specific XML tags and values.

```
# parse xml data from file using a stream
xml_context = etree.iterparse(wc_filename, events=("end",), tag="record") retrieve an entire XML "record" tag

for xml_event, xml_element in xml_context:
    # process only fully formed "record" tag end events
    # populate wc_entity
    wc_uid = xml_element.get("uid"), retrieve specific values within the tag.
    category = xml_element.get("category"), default values are supported
    text_first_name = xml_element.findtext("./person/names/" + first_name_xmltag, default="")
    text_last_name = xml_element.findtext("./person/names/" + last_name_xmltag, default="")
```

Name Screener

Name screening functionality is implemented in “namescreening.py”.

Minimizing Memory Use

The entire World-Check list needed to be loaded into memory for fast matching. Since the list can be very large, we minimized memory use by:

- Only loading the most essential values
- Streaming/buffering the values retrieved from the database

See NameScreener.load_wc_to_mem() for details.

```
def load_wc_to_mem(self): 2+ usages  louis cheng
    # clear existing data if any, we want to reload from DB
    self.clear_wc_from_mem()

    # we need to load the full name list into memory for quick string matching
    # 1) since the list is large, we need to stream/buffer the values coming in from DB
    # 2) also only load the bare essential columns
    try:
        with get_amldb_connection() as db_connection: don't buffer results - load them only when needed
            with db_connection.cursor(dictionary=True, buffered=False) as db_cursor:
                # load wc_nametype
                sql_query = "SELECT nametype_id, nametype_value FROM wc_nametype ORDER BY nametype_id" # ordered by primary key, so this is fast
                db_cursor.execute(sql_query)
                for db_row in db_cursor: load only essential values
                    self.wc_nametype.append(db_row)
```

Name Matching Algorithms

The actual name matching algorithms are implemented in:

- NameScreener.screen_ratio()
- NameScreener.screen_ratio_tokensorted()
- NameScreener.screen_extractall()

They call the rapidfuzz library.